AWS SYSTEMS MANAGER – PARAMETER STORE

Benefits of Using AWS Services

📬 AWS SQS (Simple Queue Service)

Key Benefits

- Decouples applications so producer and consumer work independently
- Enables asynchronous processing, improving performance
- Highly reliable (messages are not lost)
- Automatically scales with traffic
- Supports DLQ to handle failed messages

Business Value

- Prevents application crashes during traffic spikes
- Improves fault tolerance and resilience

🐇 Amazon MQ

Key Benefits

- Supports traditional messaging protocols (JMS, AMQP, MQTT)
- Ideal for legacy application migration to AWS
- Provides managed ActiveMQ/RabbitMQ without operational overhead
- Supports high availability (Multi-AZ)
- Ensures guaranteed message delivery

Business Value

- Enables hybrid and enterprise messaging systems
- Reduces infrastructure management effort

🔐 AWS Systems Manager Parameter Store

Key Benefits

- Centralized storage for configuration and secrets
- Supports SecureString encrypted with KMS
- Eliminates hardcoded credentials
- Fine-grained access using IAM
- Fully auditable with CloudTrail

Business Value

- Improves security and compliance
- Simplifies configuration management across environments

🛡️ AWS WAF (Web Application Firewall)

Key Benefits

- Protects applications from OWASP Top 10 attacks
- Provides rate limiting to prevent DDoS attacks
- Integrates with ALB, API Gateway, CloudFront
- Custom and managed rule support
- Real-time monitoring and logging

Business Value

- Enhances application security
- Prevents data breaches and service abuse

1. **SSM PARAMETER STORE – TASK 1**

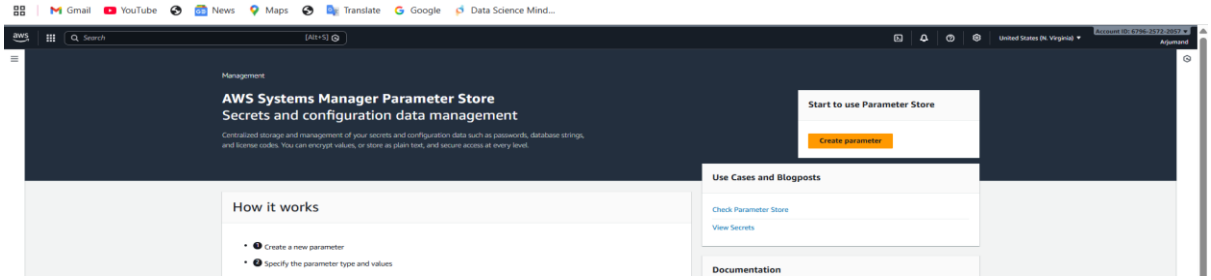
Store Application Configuration (String Parameter)

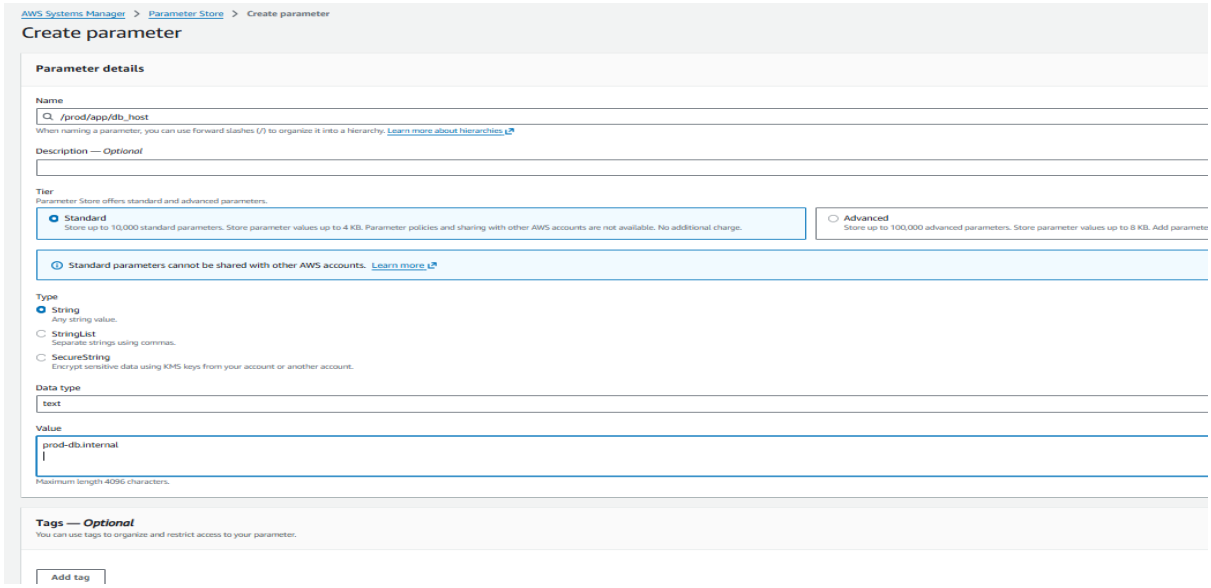
✅ **STEPS I FOLLOWED (You can say this confidently)**

1. Opened AWS Systems Manager → Parameter Store and created a new parameter.

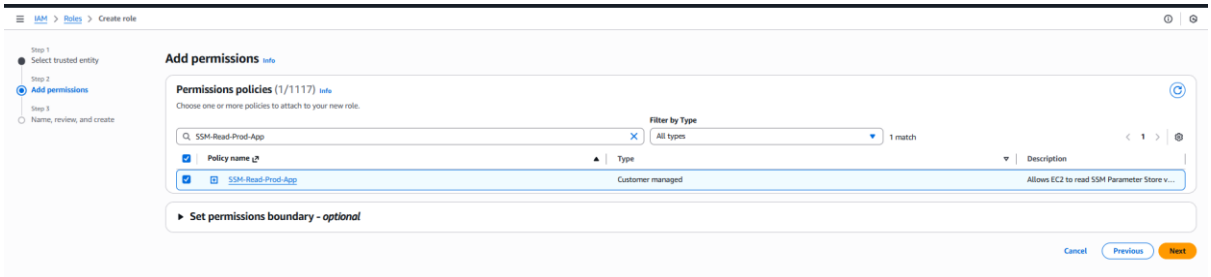
2. Used hierarchical naming /prod/app/db_host to separate environment and application.
3. Chose String type since the value is non-sensitive.
4. Created a custom IAM policy allowing ssm:GetParameter on /prod/app/*.
5. Attached the policy to an EC2 IAM role and associated the role with the EC2 instance.
6. Verified access by retrieving the parameter using AWS CLI from EC2.

Created parameter store AWS system manger > parameter store < create parameter

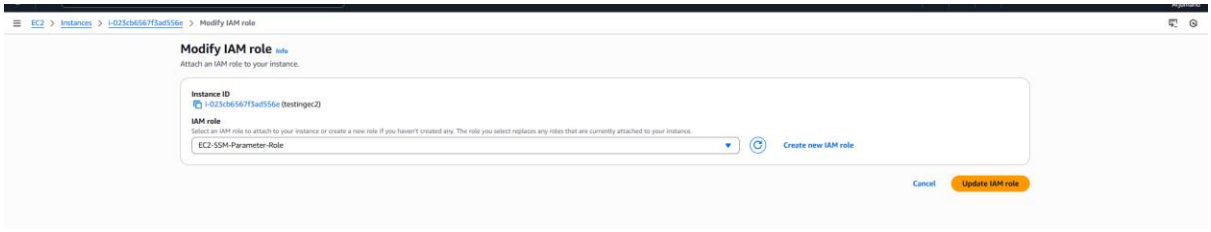




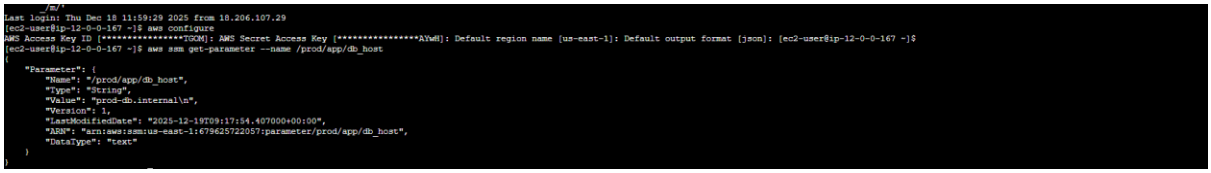
Created a custom IAM policy allowing ssm:GetParameter on /prod/app/*.



Created role and attached that policy which I have created



Role is added to ec2 and connected ec2 and checked using the command ‘aws ssm get-parameter --name /prod/app/db_host’ after aws configure



2. SSM Parameter Task 2: Secure Secrets Using SecureString + KMS

Business Scenario

Passwords and tokens must be encrypted.

Task

1. Create *SecureString* parameter

2. Encrypt using *Customer-Managed KMS key*

3. Restrict access via IAM policy

4. Retrieve decrypted value in EC2/Lambda

5. Audit access using CloudTrail

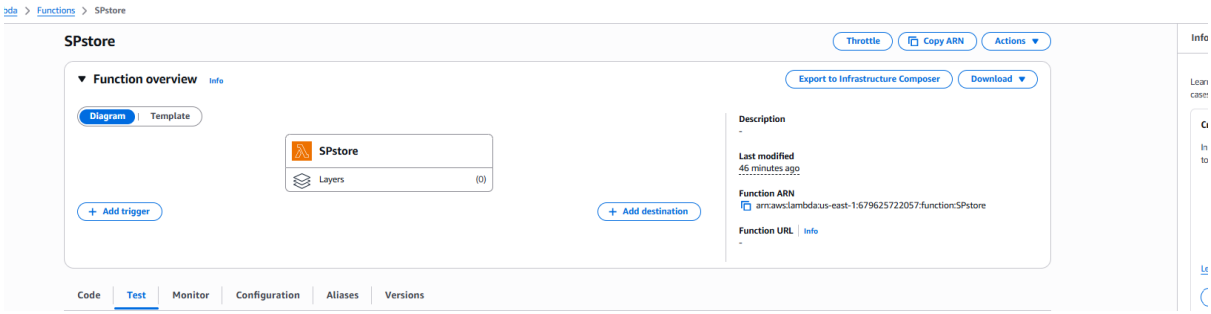
Steps:

SSM Parameter Task 2: Secure Secrets Using SecureString + KMS (10 Steps)

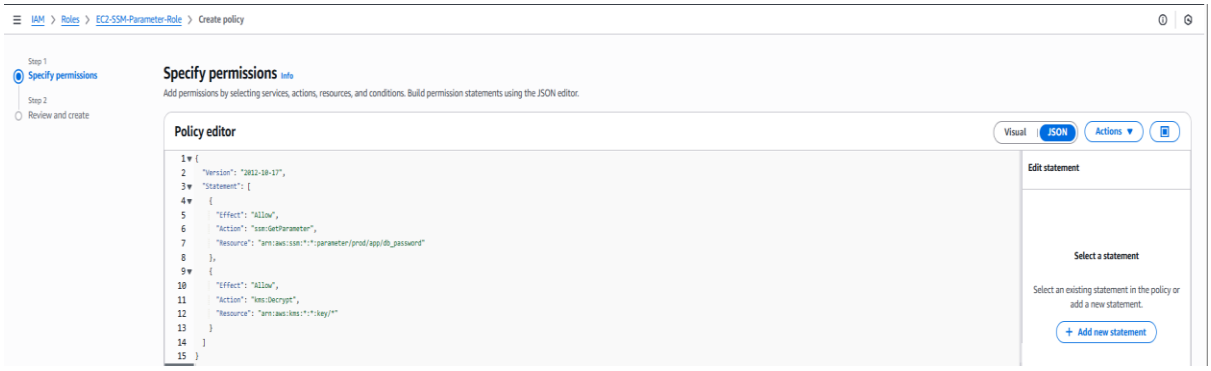
1. Created a **Customer-Managed KMS key** and enabled automatic rotation.
2. Opened **AWS Systems Manager → Parameter Store**.
3. Created a parameter named **/prod/app/db_password**.
4. Selected **SecureString** as the parameter type.

5. Chose the **customer-managed KMS key** for encryption.
6. Saved the secret value securely in Parameter Store.
7. Created a **Lambda execution IAM role** with least-privilege permissions.
8. Attached permissions for **ssm:GetParameter** and **kms:Decrypt**.
9. Updated the **KMS key policy** to allow the Lambda role to decrypt.
10. Tested the Lambda and verified success in **CloudWatch Logs**.

Created a lambda function



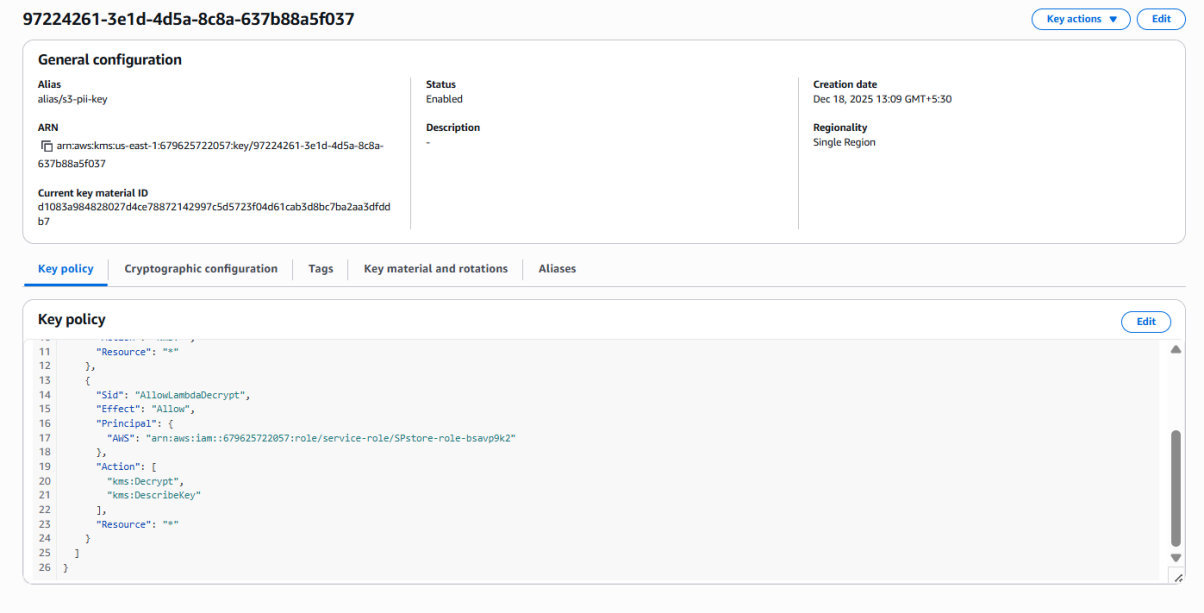
"The IAM role defines exactly what my Lambda function can do. It allows Lambda to read a specific SecureString parameter from SSM, decrypt it using KMS, and write logs to CloudWatch—nothing more."



The KMS key policy controls *who is allowed to use the encryption key itself*.

Even if IAM allows an action, **KMS will deny it unless the key policy also allows it**.

So, this policy is the **final gatekeeper**.



This proves **everything worked perfectly**:

✓ Lambda executed
✓ IAM role permissions are correct
✓ KMS key decrypted successfully
✓ SSM SecureString was retrieved
✓ Logs written to CloudWatch

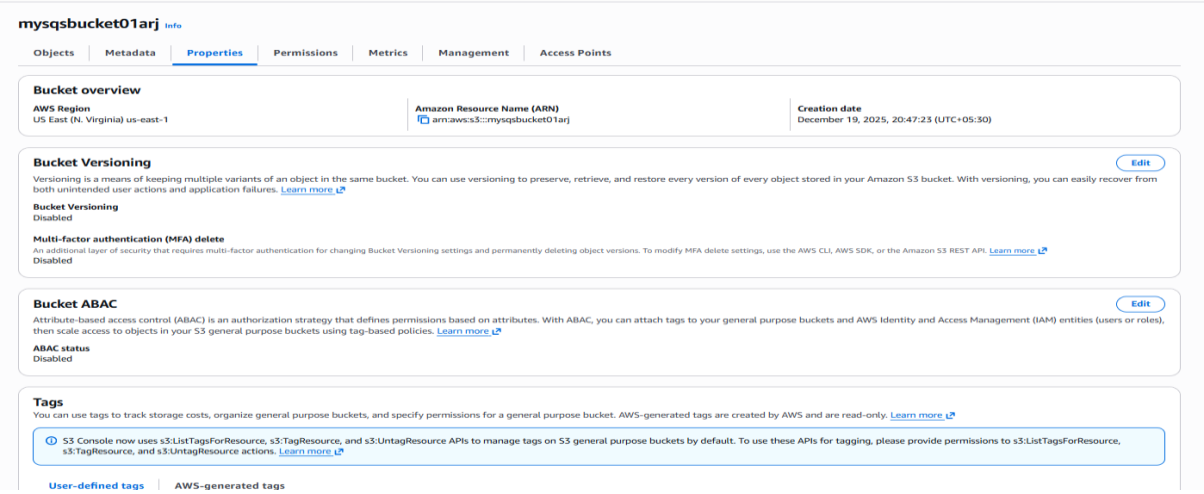
👉 **Your SSM Parameter Task 2 is 100% COMPLETE**

“I stored application secrets in SSM Parameter Store using SecureString encrypted with a customer-managed KMS key and validated secure access from Lambda using IAM roles.”

3. Setup SQS and store messages to sqs by lambda auto trigger and use s3 bucket store messages

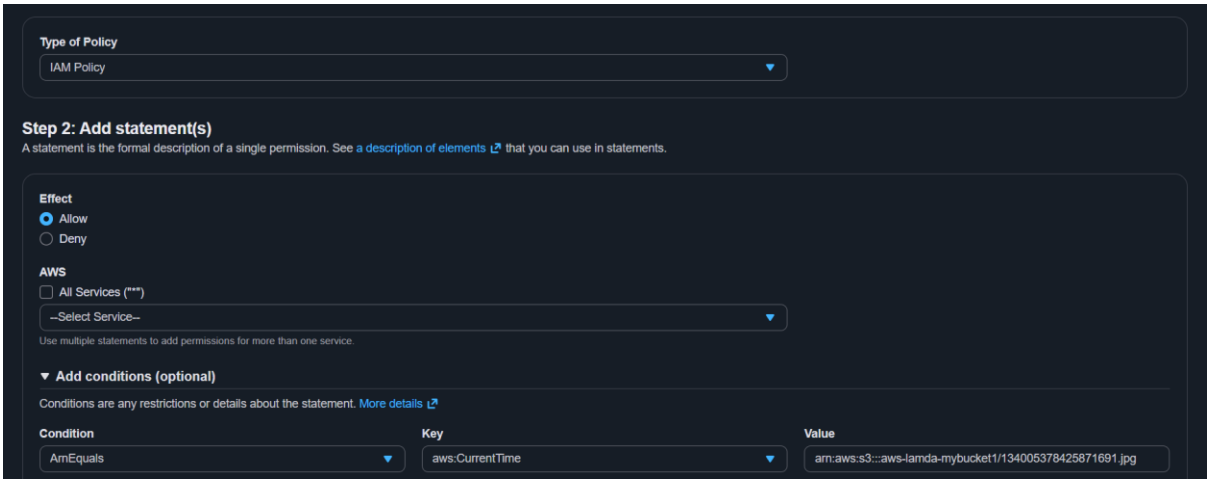
Created an S3 Bucket

- Created an S3 bucket (mysqsbucket01arj) to store objects
- Bucket is private and used as the event source
- Purpose: trigger notifications whenever a new object is uploaded



Created an SQS Standard Queue

- Created an SQS standard queue (myQueue1)
- This queue is used to decouple S3 events from Lambda processing
- Ensures reliable, asynchronous message delivery

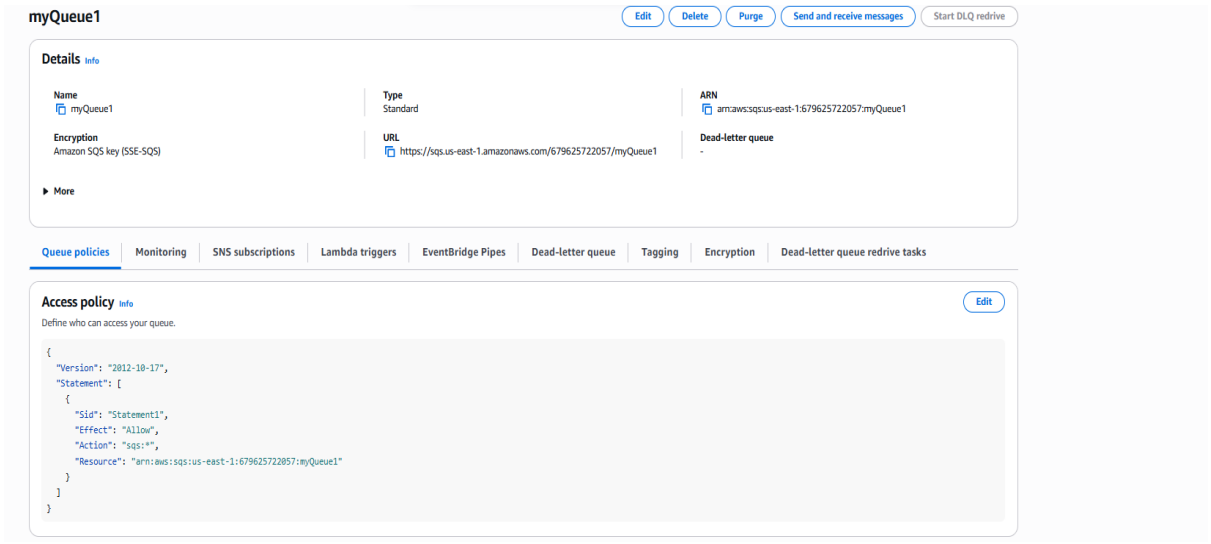


Updated SQS Access Policy (Critical Step)

- Edited the SQS queue policy to allow **only S3** to send messages
- Added a resource-based policy with:
    - Principal: s3.amazonaws.com
    - Action: sqs:SendMessage
    - Condition restricting access to the specific S3 bucket ARN
- This securely enables S3 → SQS communicatio

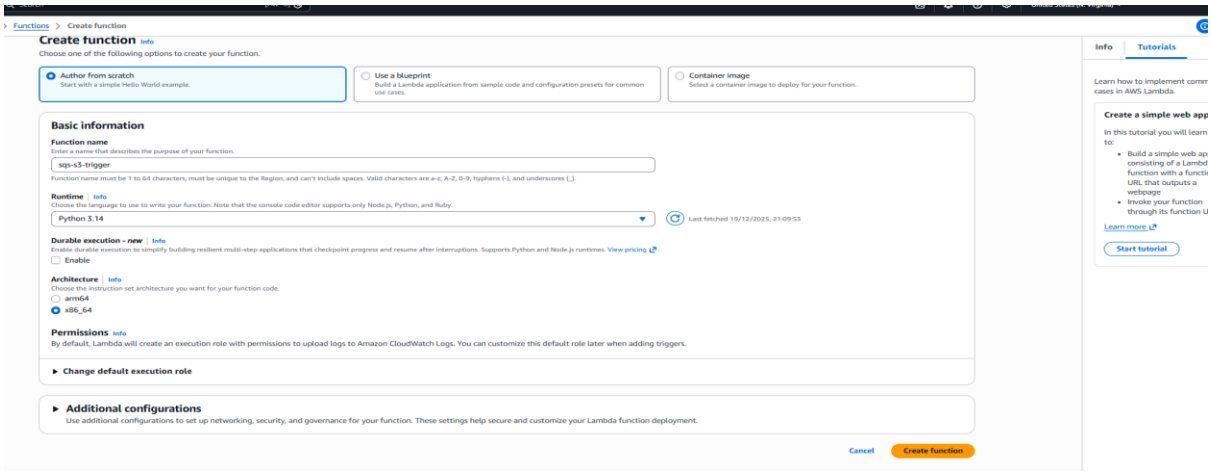
Configured S3 Event Notifications

- Enabled **ObjectCreated** events on the S3 bucket
- Configured the event destination as the SQS queue (myQueue1)
- Ensured that every new file upload sends a message to SQS



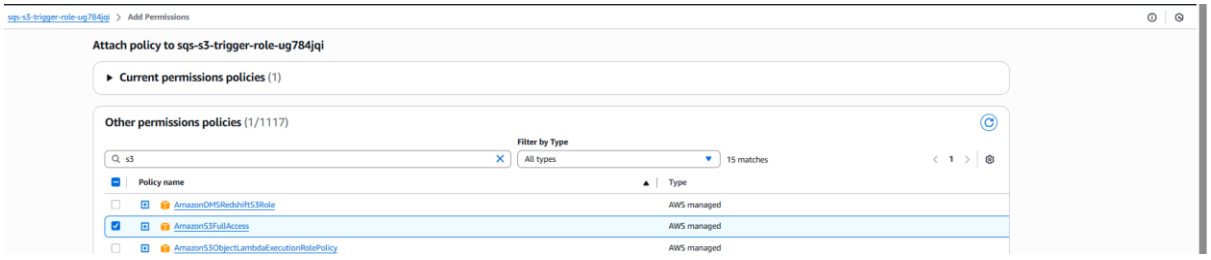
Created a Lambda Function

- Created a Lambda function using **Python runtime**
- Function purpose: process messages coming from SQS
- Lambda is used for serverless, scalable event processing



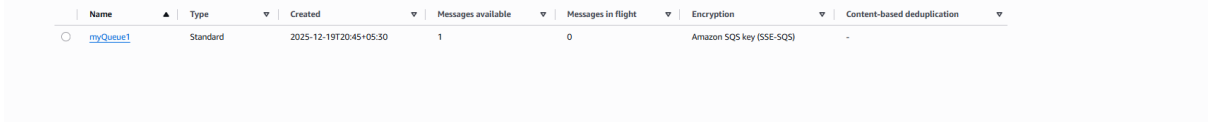
Assigned IAM Permissions to Lambda

- Attached IAM permissions to allow Lambda to:
  - Receive messages from SQS
  - Delete messages after processing
  - Read queue attributes
- Ensured least-privilege access



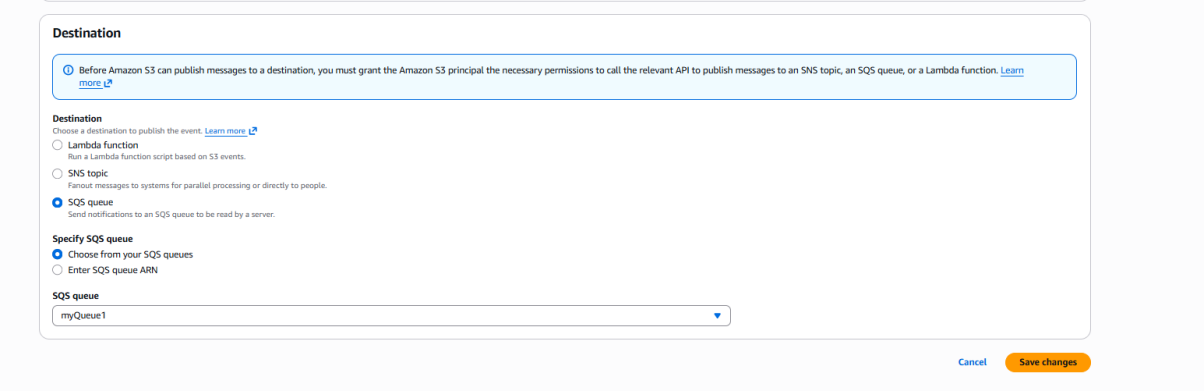
Attached SQS as a Lambda Trigger

- Added the SQS queue as an event source (trigger) for Lambda
- Enabled the trigger so Lambda automatically runs when messages arrive
- Batch processing handled by Lambda service



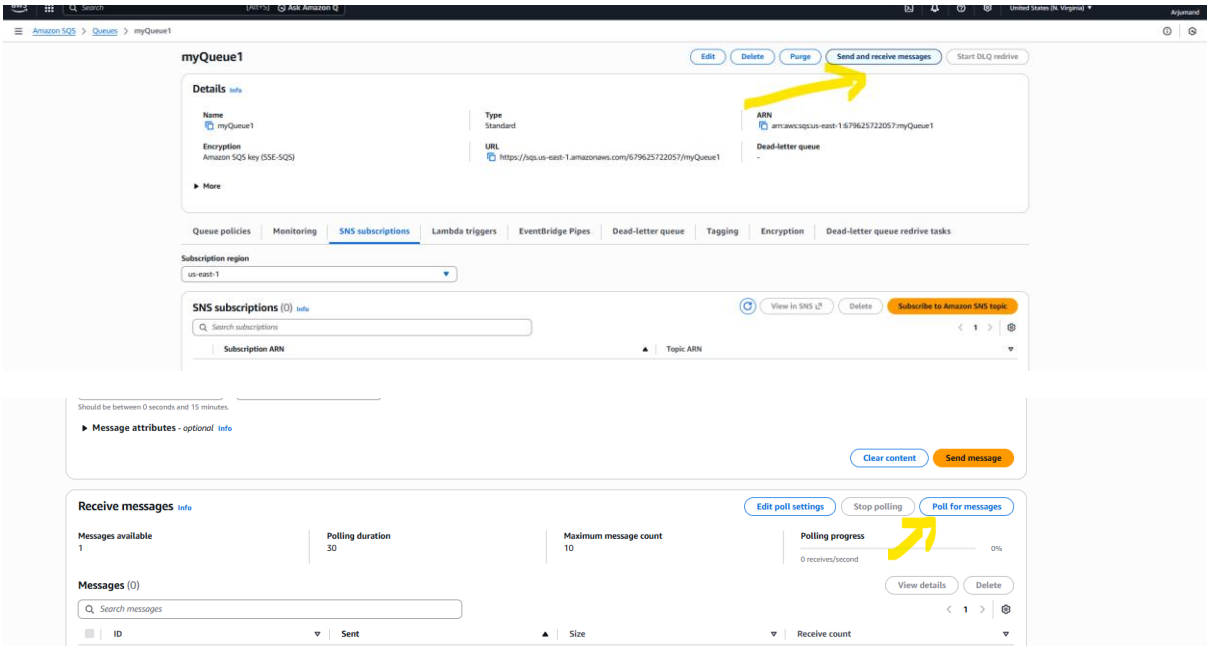
Implemented Lambda Python Code

- Lambda code parses the SQS message
- Extracts S3 bucket name and object key from the event payload
- Logs the file upload details to CloudWatch Logs

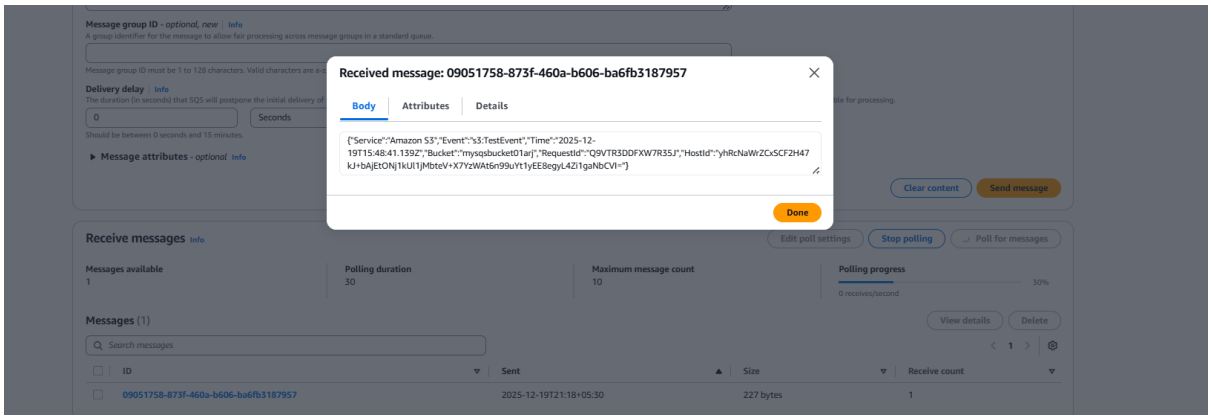


Tested the End-to-End Flow

- Uploaded a file to the S3 bucket
- Verified message arrival in SQS
- Confirmed Lambda execution in CloudWatch Logs
- Ensured successful processing without errors



Messages triggers and successfully message is displayed in SQS

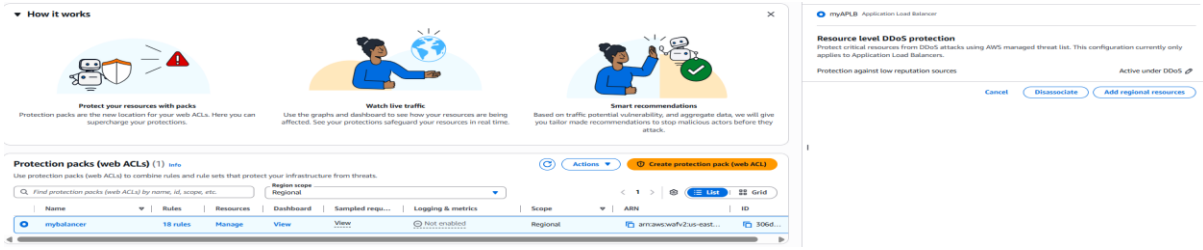


4. Create a WAF and attach resource as load balancer

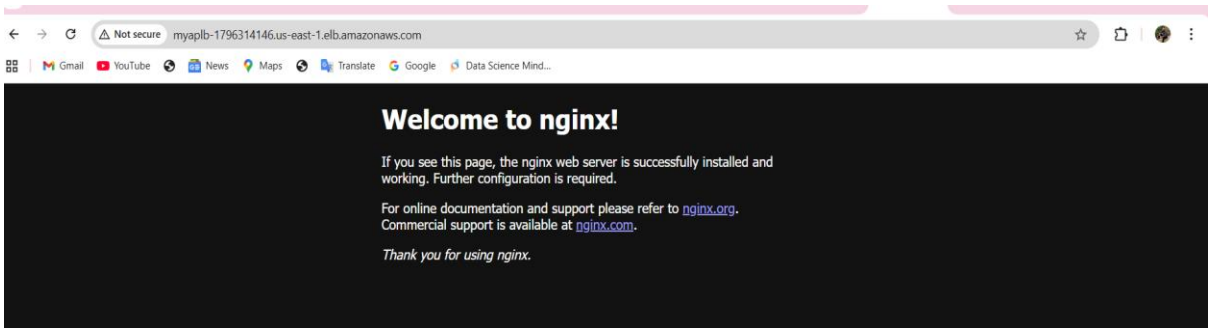
Steps Followed to Set Up AWS WAF with Application Load Balancer

1. Created an **Application Load Balancer (ALB)** and attached EC2 instances in a target group.
2. Verified ALB accessibility using the **DNS name**.
3. Opened **AWS WAF & Shield** and created a **Web ACL (Protection Pack)** with **Regional scope**.
4. Attached the Web ACL to the **Application Load Balancer**.
5. Created an **IP set** with the correct CIDR range (due to dynamic IP, used subnet range instead of /32).
6. Added a **Custom rule**:
    - Action: **Block**
    - Condition: **Originates from an IP address**
    - IP address list: Created IP set
7. Ensured the rule used **Source IP address** (not X-Forwarded-For header).
8. Placed the **Block rule at the top priority** so it evaluates before allow rules.
9. Adjusted rule order to ensure no **Allow rules override the Block rule**.
10. Tested access to ALB and confirmed blocking behavior based on WAF rules.

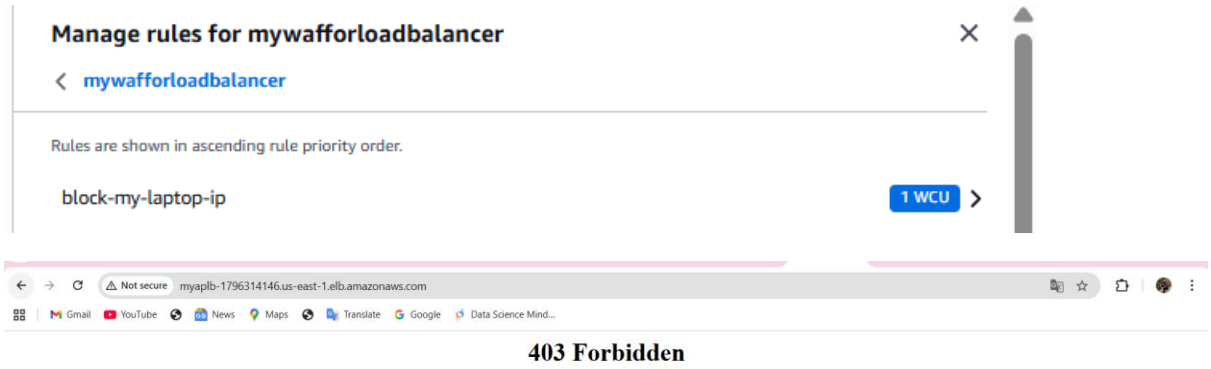
✅ Result: **AWS WAF successfully configured and protecting the Application Load Balancer.**



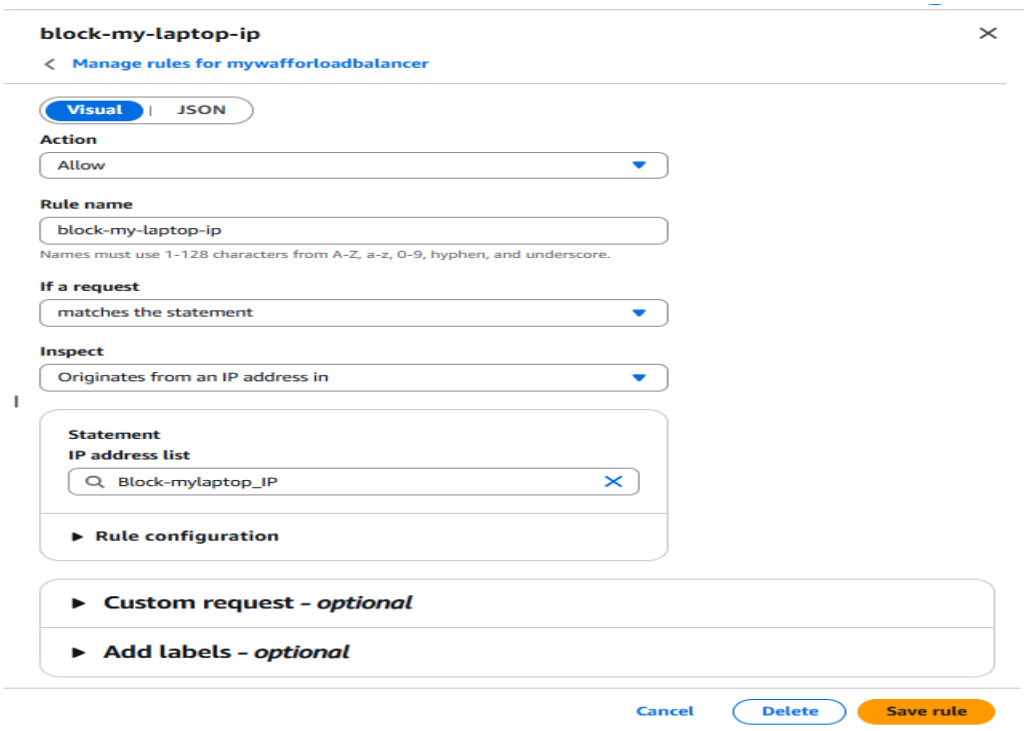
With help of WAF also I am able to access



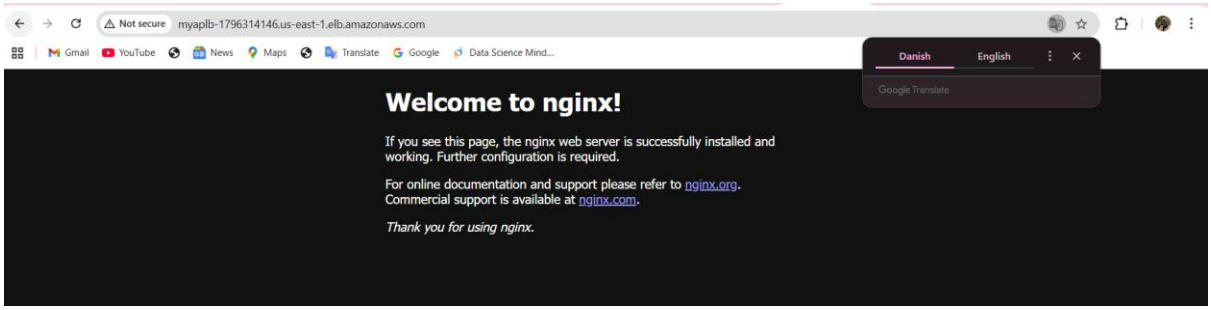
Blocked now my IP address.



Now I have allowed my ip address again



Now again it is allowing so successfully attached firewall.



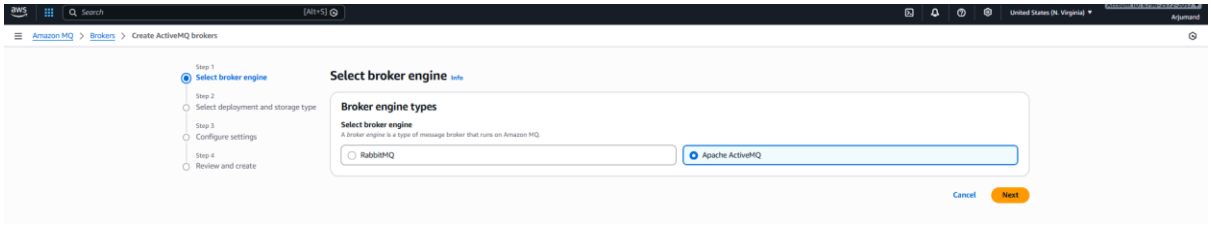
5. **Create and setup MQ broker and login**

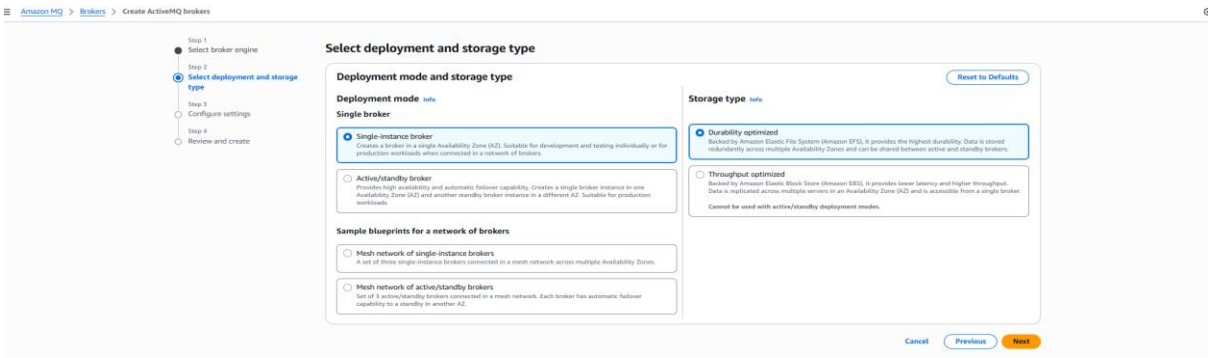
Steps Followed to Set Up Amazon MQ (ActiveMQ)

1. Opened Amazon MQ in AWS Console and clicked Create broker.
2. Selected Apache ActiveMQ as the broker engine and chose Single-instance broker for learning.
3. Provided broker name, engine version, instance type, and created broker user credentials (username & password).
4. Selected a custom VPC (default VPC was deleted) and chose two subnets from different Availability Zones.

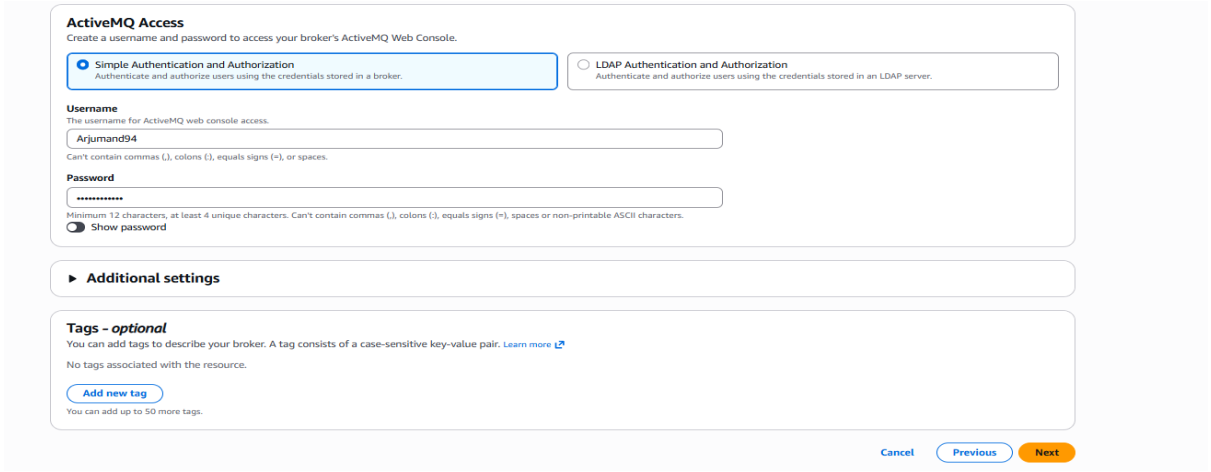
5. Attached a security group and allowed required ports (8162 for console, 61617 for OpenWire).
6. Created the broker and waited until the status changed to Running.
7. Opened the ActiveMQ Web Console URL using port 8162.
8. Logged in using the broker credentials (not IAM).
9. Verified successful setup by accessing the ActiveMQ dashboard and broker details.

✅ **Result: Amazon MQ broker setup and access completed successfully.**

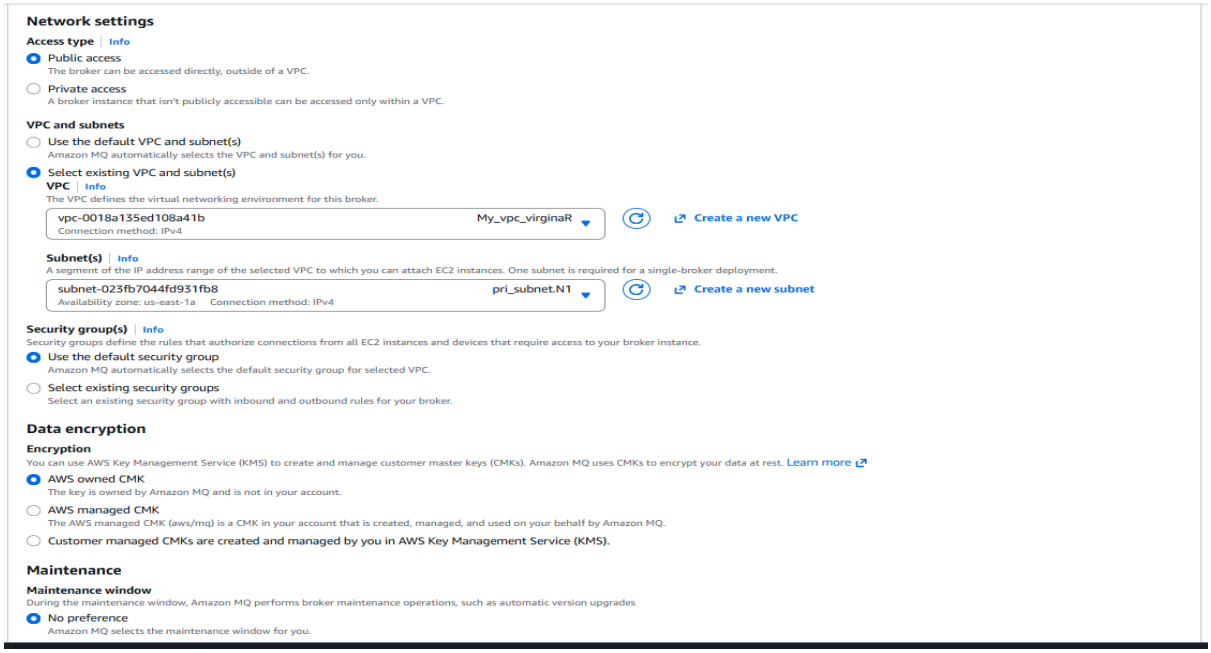




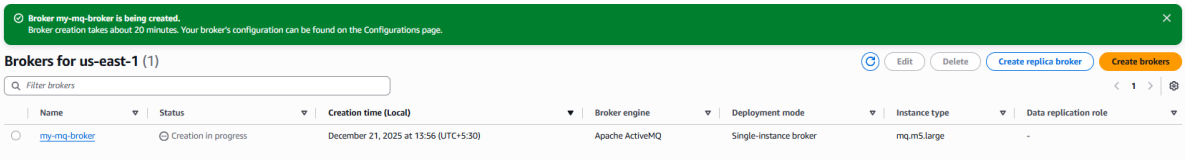
Provided username and password



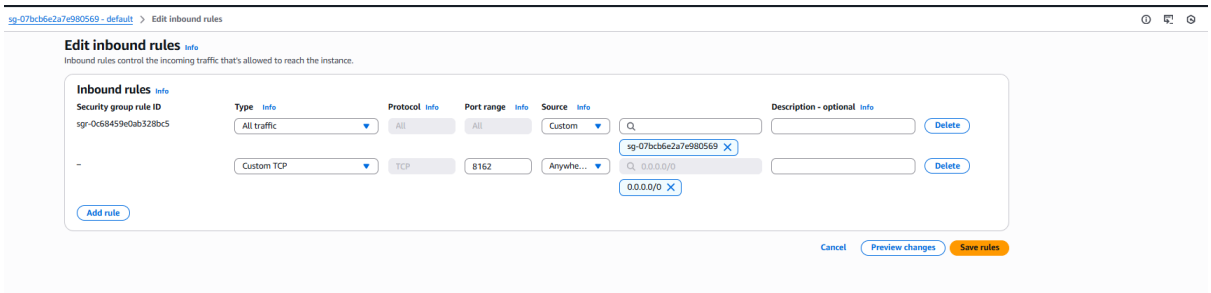
Selected default VPC and subnet



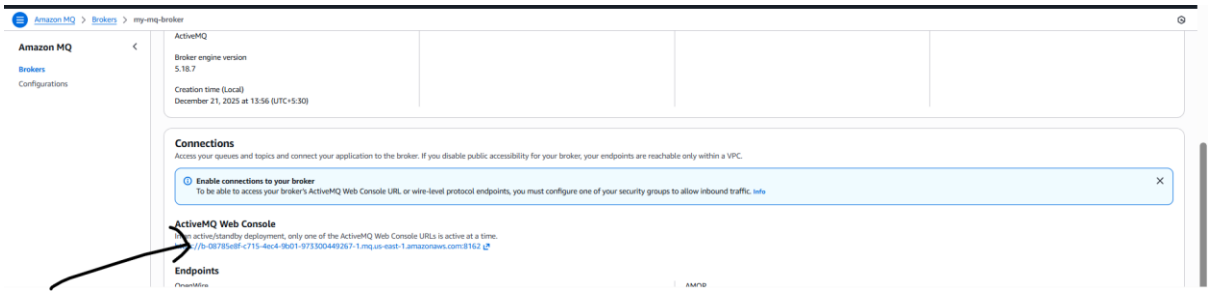
MQ created with default vpc and default subnet



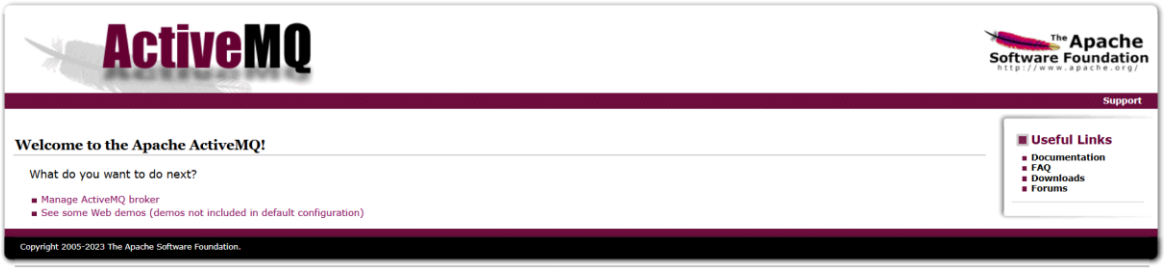
Edit security group of broker and allow tcp 8162 port



Click the link which is there in broker



After clicking the link we can see the broker page and login with the creditials which we created while creating MQ broker



Graphic Design By Hiram

After clicking Manage ActiveMQ it will ask credentials then enter then the page will open